Introduction to Computer Science

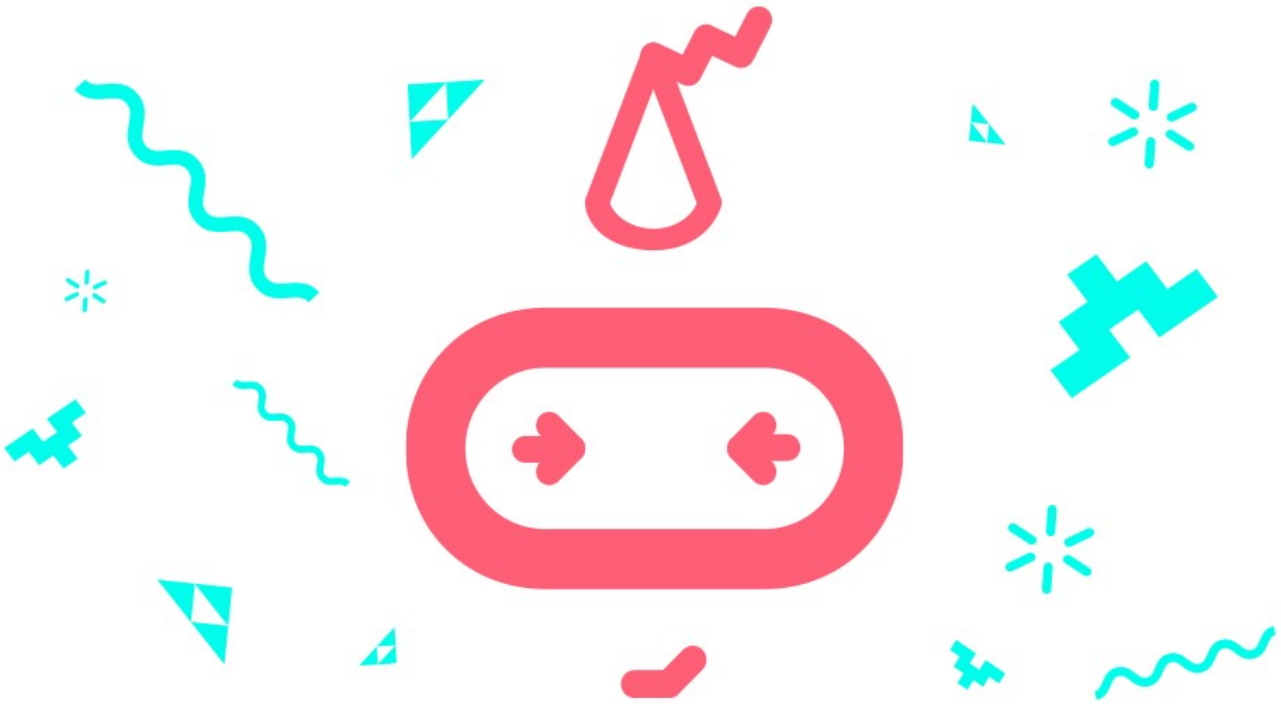
Unit 11:

Arrays

Student workbook

makecode.microbit.org

Table of Contents



Overview

Unit summary

In this unit, you will explore the concept of arrays. An array is a list of information. Previously, you learned how to use variables as a way to store information. Arrays store multiple values in a sequential order and are retrieved from a single object—the array itself. In the unplugged activity, you will learn the usefulness of arrays as a collection of related items and experience common algorithms for sorting and storing data, including bubble, selection, and insertion sorting strategies. The two coding activities will demonstrate using array operations to store and retrieve string data for a fun micro:bit version of charades and numeric data to create random constellations with the LEDs on the micro:bit screen. In this unit's project, you will be challenged to craft a musical instrument that uses arrays to store sequences of notes to play on the micro:bit when an input occurs.

Learning goals

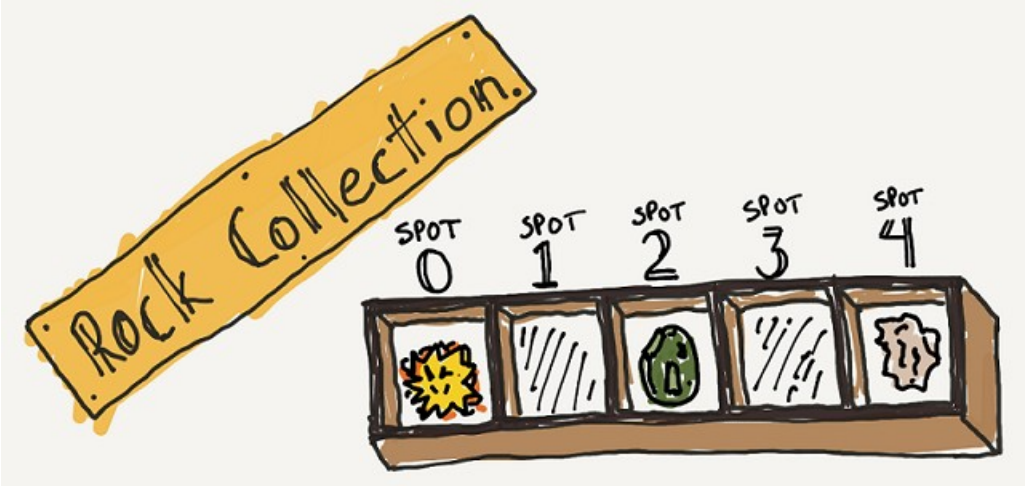
During this unit, you will:

- Explain the steps you would take to sort a series of numbers.
- Recognize three common sorting algorithms.
- Practice creating arrays.
- Practice storing and retrieving values in arrays.
- Learn common array operations such as setting and getting values by index.
- Demonstrate understanding of arrays and apply skills by creating a musical instrument that uses a micro:bit and a program that correctly and effectively uses arrays to store data.

Lesson A: Understanding arrays

Overview: Arrays

Any collector of coins, fossils, or baseball cards knows that at some point you need to have a way to organize everything so you can find things. For example, a rock collector might have a tray of specimens numbered like this:

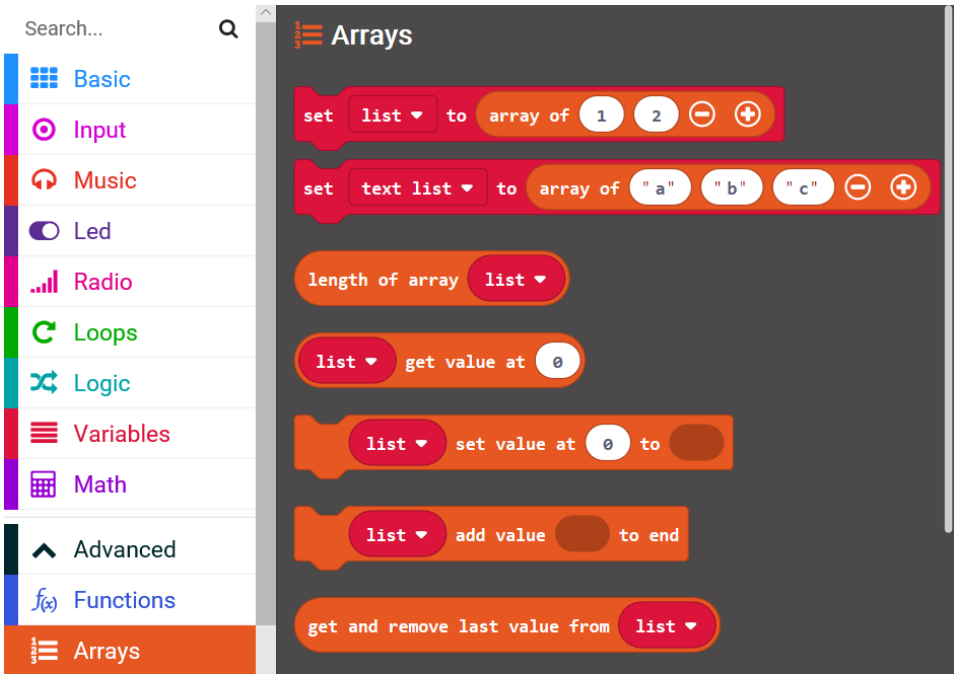


Every rock in the collection needs its own storage space and a unique address so you can find it later.

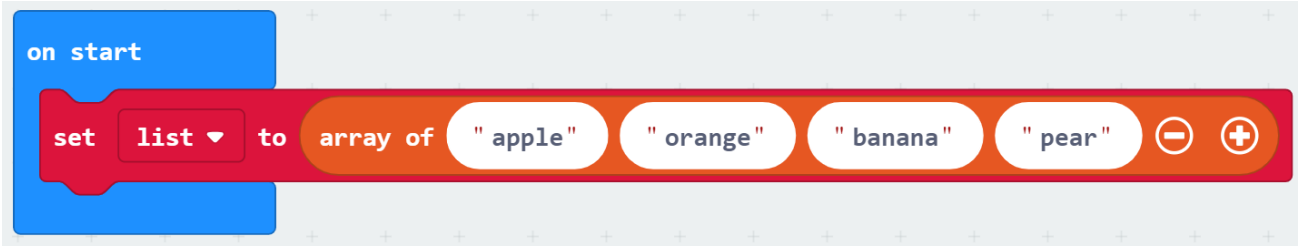
As your MakeCode programs get more and more complicated and require more variables to keep track of things, you will want to find a way to store and organize all of your data. MakeCode provides a special category for just this purpose called an array which is essentially just a list, or a collection, of similar things.

- Arrays can store numbers, strings (words), or sprites. They can also store musical notes. But they must store values of a similar type—an array cannot contain both numbers and words.
- Every spot in an array can be identified by its index, which is a number that corresponds to its location in the array. The first slot in an array is index 0, just like the rock collection above.
- The length of an array refers to the total number of items in the array, and the index of the last element in an array is always one less than its length (because the array index numbering starts at zero.) So, in the Rock Collection above, the length of the array is 5 (it can hold 5 rocks), and the index of the last element is 4.

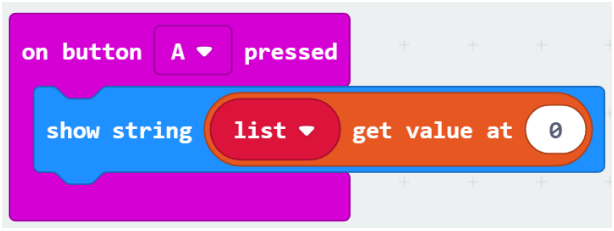
In MakeCode, you can create an array by assigning it to a variable. The Array blocks are found under the Advanced Toolbox menu.



This code creates an empty array called list, then fills it with four fruits as text or string values, indexed from 0 to 3. The index of the first value (apple) is 0. The index of the second value (orange) is 1. The index of the last value (pear) is 3.



You can get items out of the array by specifying its index. In this example, when you press Button A, the microbit will show the word “apple” on the screen, which is the string value located in index 0 of the list array.



Solution link: makecode.microbit.org/_K4523f8JTCfD

There are lots of other blocks in the Arrays Toolbox drawer. The next few activities will introduce you to them.

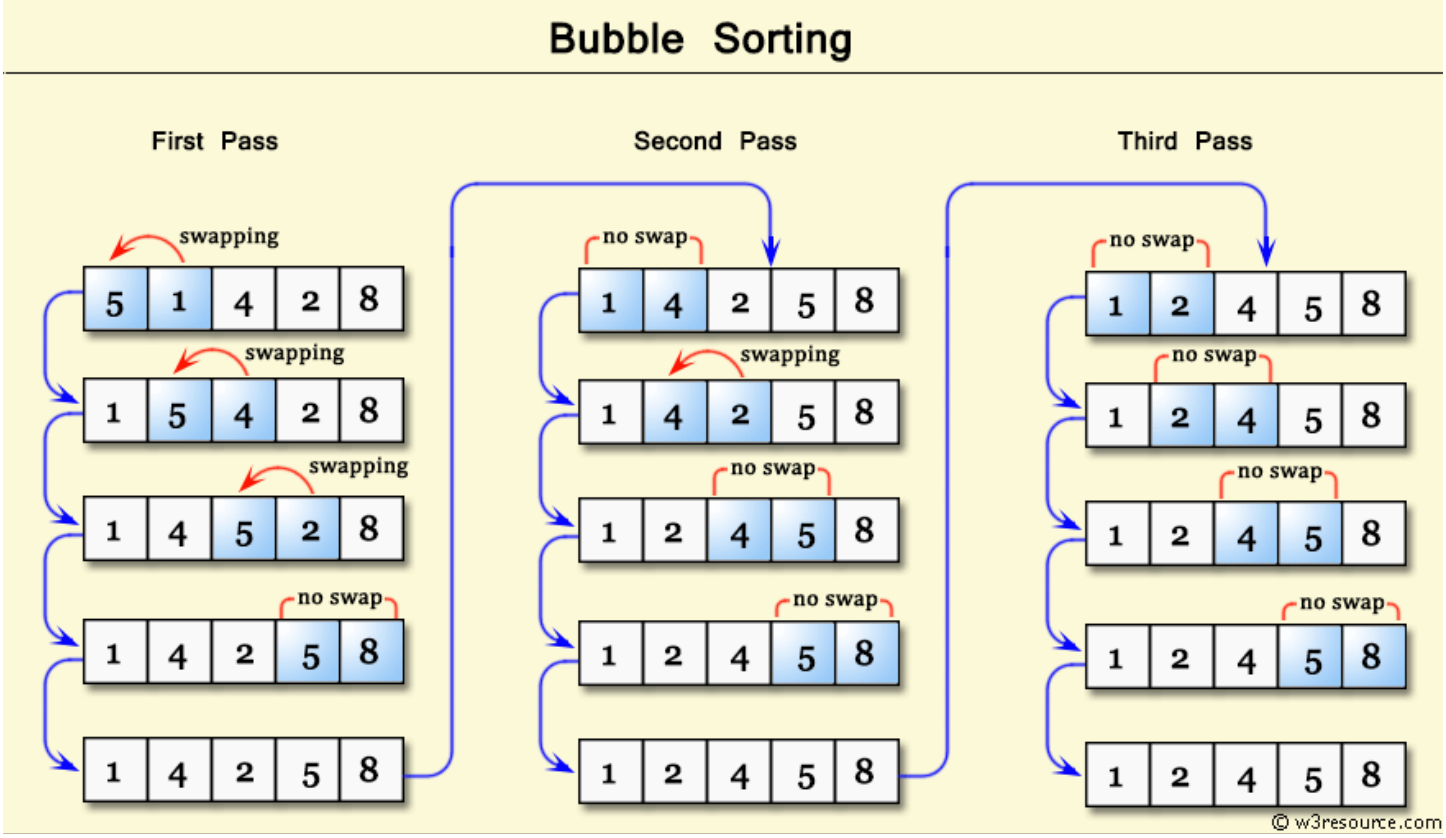
Some Different Types of Sorts

In computer science, there are certain common strategies, or algorithms, for sorting a collection of values.

- Bubble sort
- Selection sort
- Insertion sort

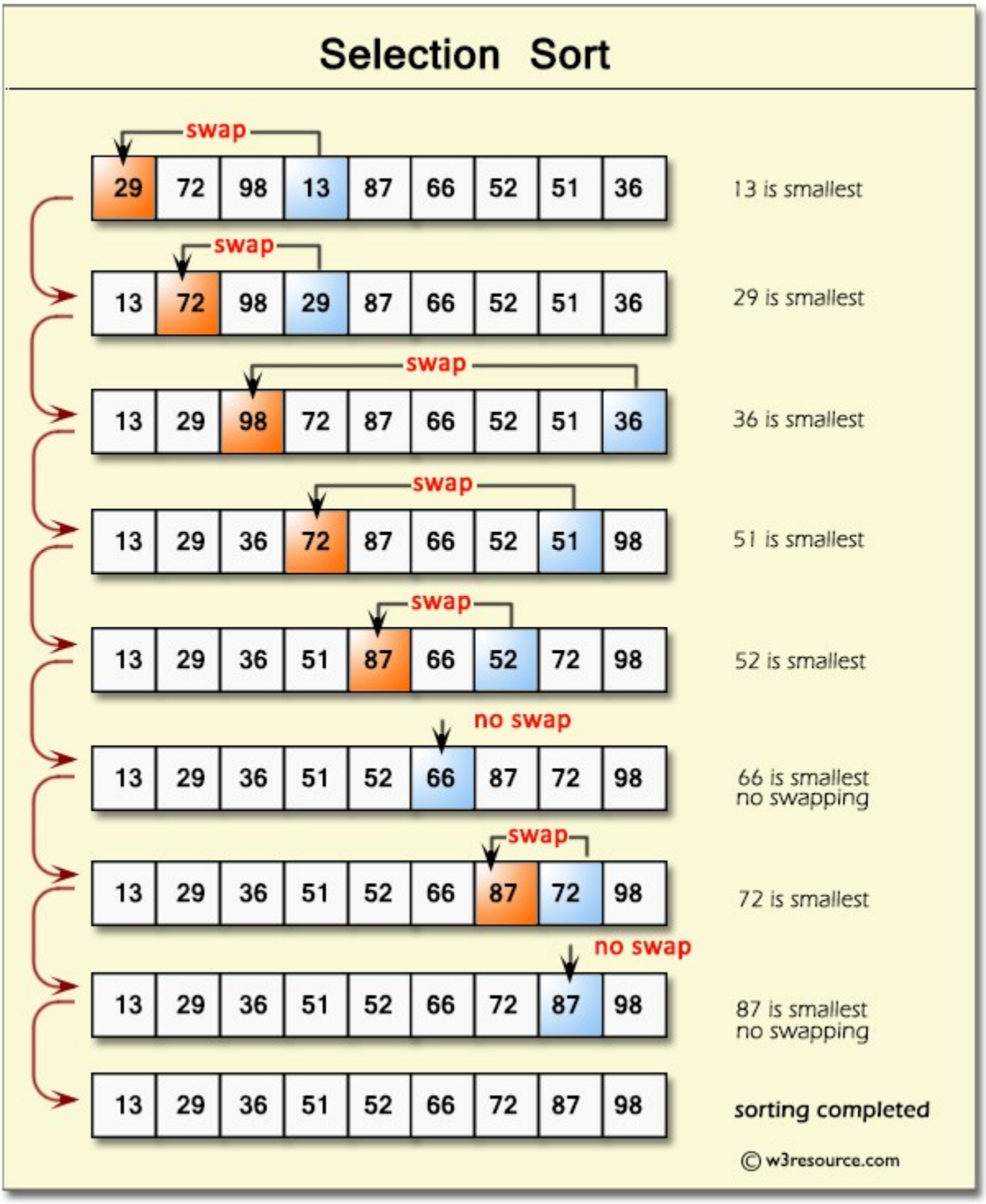
Bubble sort

In a bubble sort, each consecutive pair of values is compared and the larger value is swapped to the right. As multiple passes over the array occur, the larger values “bubble” up towards one end, like bubbles in a fizzy pop. Because you have to compare the same pairs of numbers repeatedly, bubble sort is not terribly efficient—although it does have the advantage that if you make a complete pass comparing every consecutive pair and no swaps occur, you can preemptively declare the array sorted and your task is done.



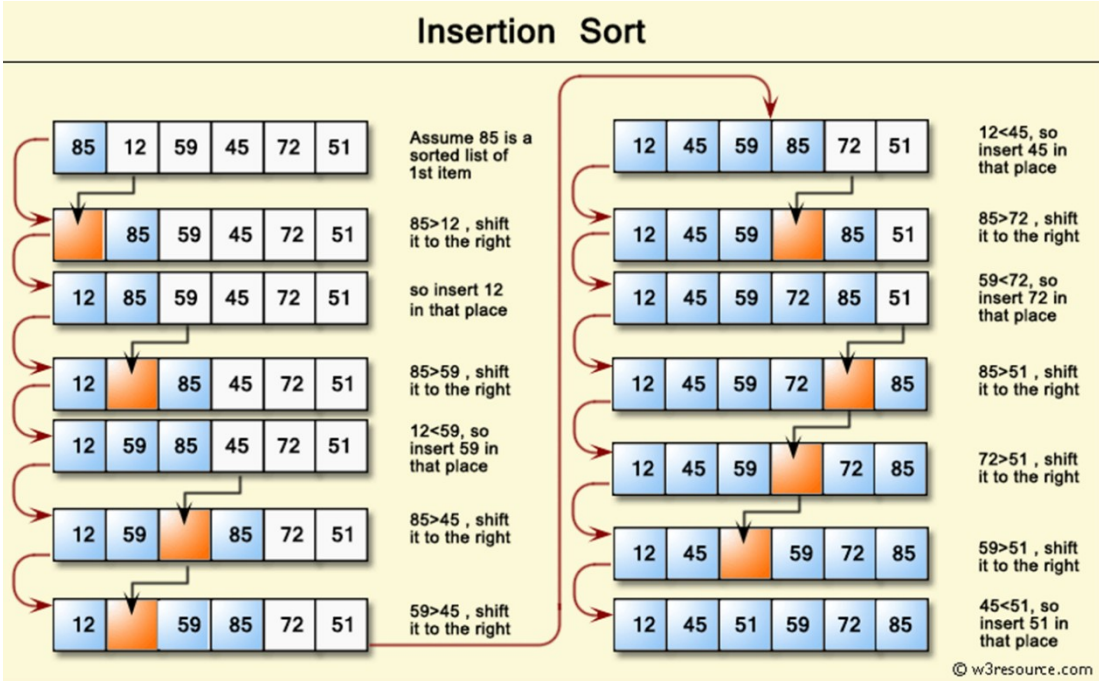
Selection sort

In a selection sort, multiple passes over the array are made to determine the smallest number on that pass. That smallest number is then swapped with the number on the end and the next pass over the remaining unsorted numbers occurs. Because there is no way to tell if you have finished early, selection sort will always take the same number of passes as the number of elements in the array to complete. So, it is even slower than bubble sort, on average.



Insertion sort

Imagine that we're sorting a pile of papers alphabetically. We might place the top paper to the side, starting a new pile, and consider it sorted. Then we would take the next paper and place it either before or after the previous paper in the sorted pile, depending on whether that paper comes before it or after it in the alphabet. Every subsequent paper that comes off the unsorted pile, we would place right where it belongs in the sorted pile. That way, we only touch each paper once, and after one pass through the array, we are done. At first this seems more efficient than bubble sort and selection sort, but as the sorted pile grows larger, the number of comparisons we have to make to place each paper in the right place also increases. So, insertion sort actually isn't any more efficient than the other two methods, although it is probably closest to the way a human being would sort an array of elements.



Lesson B: Coding with arrays

Overview of the coding activities

You'll do three activities today to code arrays:

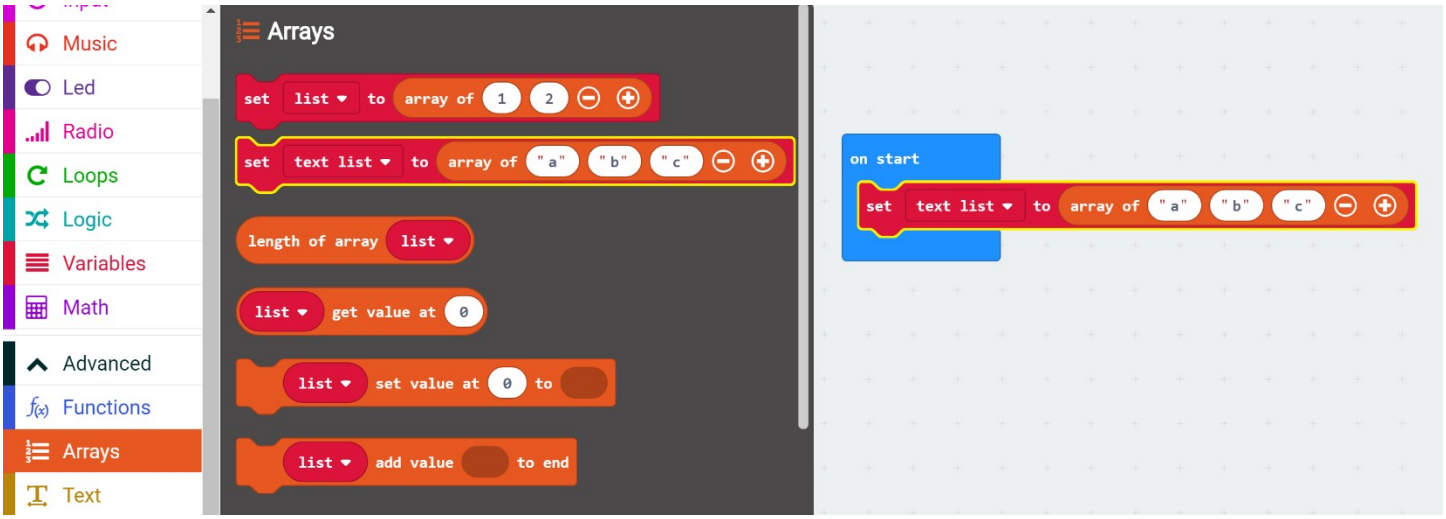
- Headband charades – using arrays that store string elements (words)
- Starry, starry night – using arrays that store numeric elements
- Traversing an array – using a special loop just for arrays

Coding activity 1: Headband charades

Create an array of words that can be used as part of a charades-type game. This activity is based on a very popular phone app invented by Ellen DeGeneres (bits.blogs.nytimes.com/2013/05/03/ellen-degeneres-iphone-game).

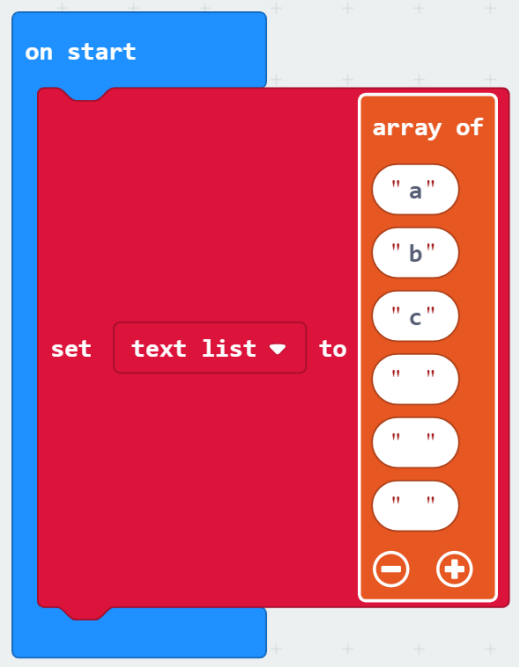
Set the arrayWords

1. In Microsoft MakeCode, start a new project and name it something like ‘charades’. Either delete the ‘forever’ block in the coding Workspace or move it to the side as it’s not used in the activity.
2. From the Arrays Toolbox drawer, drag a ‘set (text list0)’ block onto the Workspace and drop it into the ‘on start’ block.



Notice that the array comes with three string blocks. You’ll want more for your charades game.

3. Select the **(+)** symbol at the end of the ‘array of’ oval block. Add as many values (elements) as you’d like to the array block by continuing to select the **(+)**. For now, you’ll add four more values for a total of six values.



4. Fill each string with one word. Choose words that will be fun for a game of charades. For example:

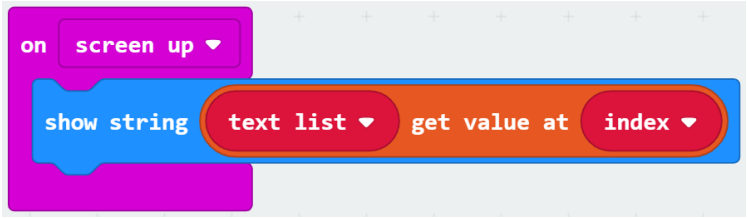
Code ‘on (screen up)’

Now, you need a way to access one word at a time from this array of words.

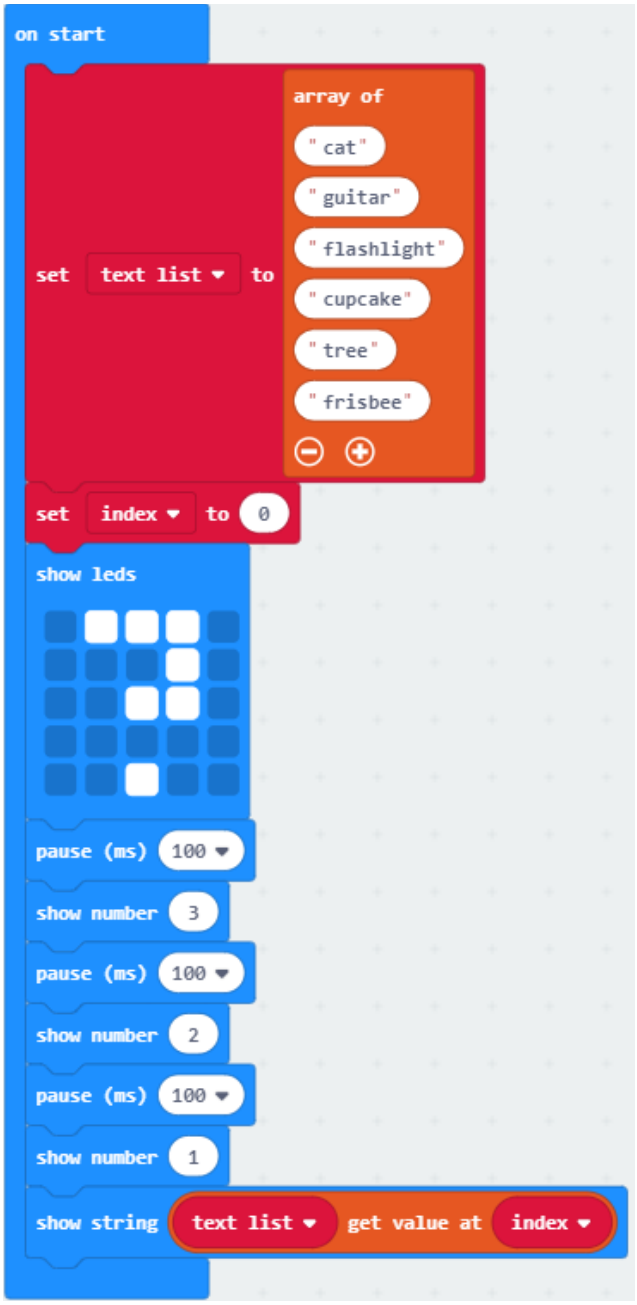
5. You can use the ‘show string’ block from the Basic Toolbox drawer and the ‘on screen up’ event handler from the Input Toolbox drawer (this is a dropdown menu choice of the ‘on shake’ block) to tell the micro:bit to display a word when you tilt the micro:bit up.

For this version, you’ll display the words one at a time in the order they were first placed into the array.

6. You’ll use the index of the array to keep track of what word to display at any given time, so you’ll need to create an ‘index’ variable using the Make a Variable button in the Variables Toolbox drawer.



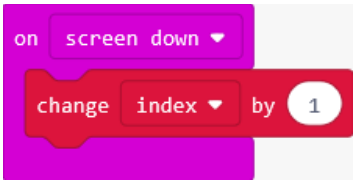
7. To start the game with the index at zero, add a ‘set’ variable block to the ‘on start’ block.
8. Next, add the following:
    a. An image as a placeholder for when the program has started. Since charades is a guessing game, make one that looks like a question mark (?)
    b. A countdown to the first word using show number blocks and pause blocks
    c. And show the first word in the array



Code ‘on (screen down)’

So far you have a start to your game and a way to display the first word. Once that word has been guessed (or passed), you need a way to advance to the next word in the array.

9. You can do this by changing the index of the array with the ‘on screen down’ event handler from the Input Toolbox drawer (this is a dropdown menu choice of the ‘on shake’ block) to advance to the next word when you tilt the micro:bit down.



You have a limited number of elements in your array, so to avoid an error, you need to check and make sure you are not already at the end of the array before you change the index.

10. Under the Arrays Toolbox drawer, drag out a ‘length of’ block. The ‘length of’ block returns the number of items (elements) in an array. For your array, the length of block will return the value 6. But because computer programmers start counting at zero, the index of the final (6th) element is 5.

Some pseudocode for your algorithm logic:

When the player places the micro:bit screen down:

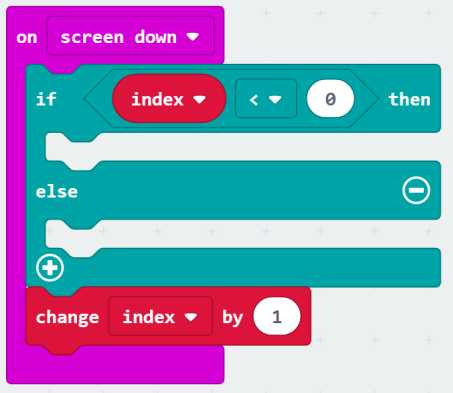
a. Check the current value of the index.
b. If the current value of the index is less than the length of the array minus one (see array bounds note),
   Then change the value of the index by one,
   Else indicate that it is the end of the game.

Array bounds

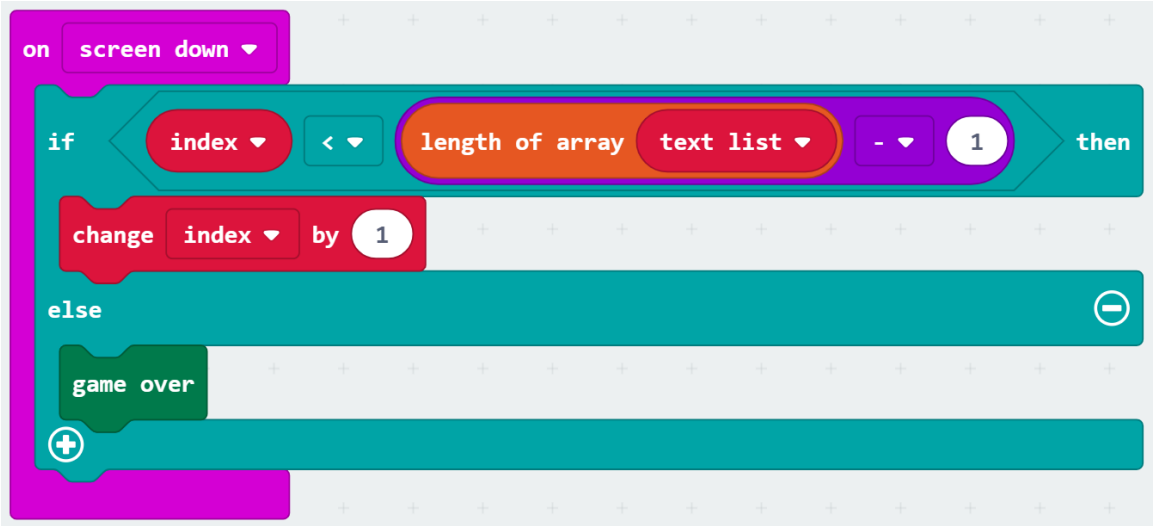
Your array has a length 6, so this will mean that as long as the current value of the index is less than 5, you will change the array by one.

Using ‘less than the length of the array minus one’ instead of the actual numbers for your array makes this code more flexible and easier to maintain. You can easily add more elements to your array and not have to worry about changing numbers elsewhere in the code.

11. From the Logic Toolbox drawer, drag out an ‘if…then…else’ block onto the Workspace and drop it into the ‘on screen down’ block.
12. From the Logic Toolbox drawer, drag a ‘0<0’ comparison block onto the Workspace and drop it into the ‘if…then’ clause replacing the default value of ‘true’.
13. From the Variables Toolbox drawer, drag an ‘index’ variable block onto the Workspace and drop it into the first slot of the comparison block



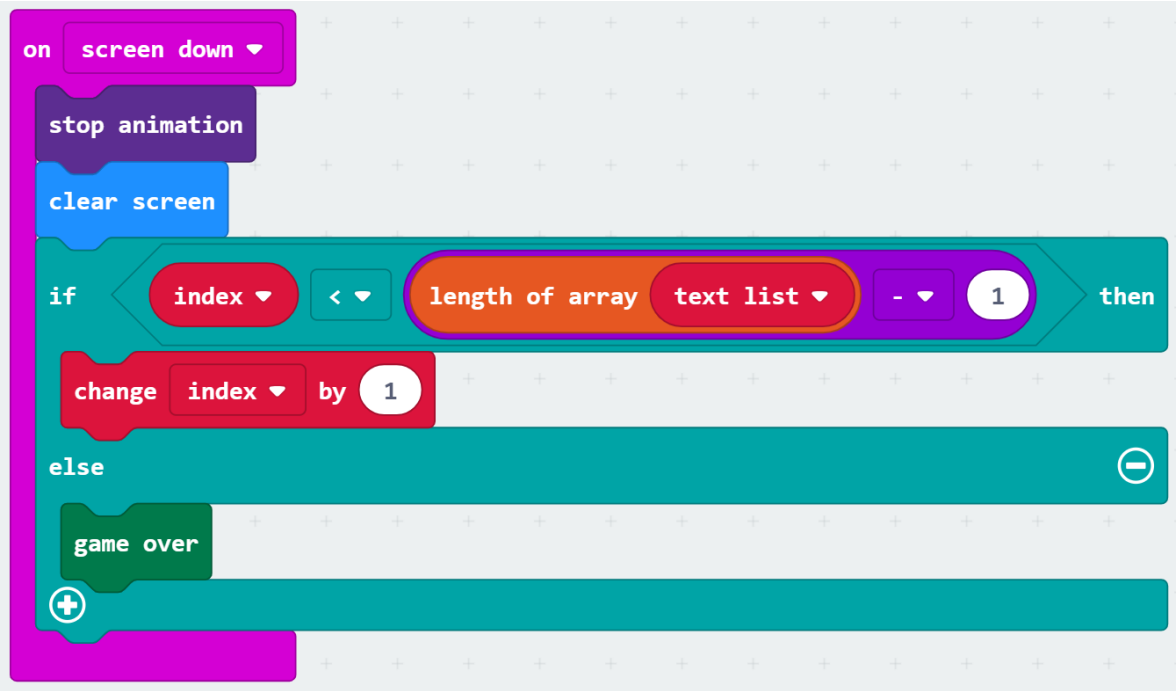
14. You need to check that the current index value is less than the length of the array minus one (the last index value).
    a. From the Math Toolbox drawer, drag a ‘0-0’ operator block onto the Workspace and drop it into the second slot of the comparison block.
    b. From the Array Toolbox drawer, drag a ‘length of array’ block onto the Workspace and drop it into the first slot of the ‘0-0’ math operator block.
    c. In the ‘length of array’ block, use the dropdown menu to select the ‘text list’ array.
    d. In the second slot of the math operator block, type 1.
    e. Drag the ‘change index’ block from below the ‘if…then…else’ block into the ‘then’ clause.
    f. From the Game Toolbox drawer (under the Advanced Toolbox menu), scroll down to find the ‘game over’ block. Drag it onto the Workspace and drop it into the ‘else’ clause.



Add some polish

To make your game more polished, you'll add two more blocks for smoother game play.

15. In case a word is already scrolling on the screen when a player places the micro:bit screen down, you can stop this animation and clear the screen for the next word by using a 'stop animation' block from the Led More Toolbox drawer, and a 'clear screen' block from the Basic…More Toolbox drawer.



Play the game

Download the program to the micro:bit and play in pairs or small groups.

There are different ways you can play charades with your program. Here is one way you can play with a group of friends.

a. With the micro:bit on and held so Player A cannot see the screen, another player starts the program to see the first word.
b. The other players act out this word charades-style for Player A to guess.
c. When Player A guesses correctly or decides to pass on this word, a player places the micro:bit screen down.
d. When ready for the next word, a player turns the micro:bit screen up. Play continues until all the words in the array have been used.

Mod this!

- Add a headband to hold the micro:bit on the Players' foreheads (using cardboard, paper, rubber bands, etc.)
- Add a way to keep score
- Keep track of the number of correct guesses and passes
- Add a time limit

Coding activity 2: Starry, starry night

In this micro:bit activity, you will create a set of random constellations on the micro:bit screen. You will use an array filled with numbers to tell you how many stars (dots) should be in each constellation.

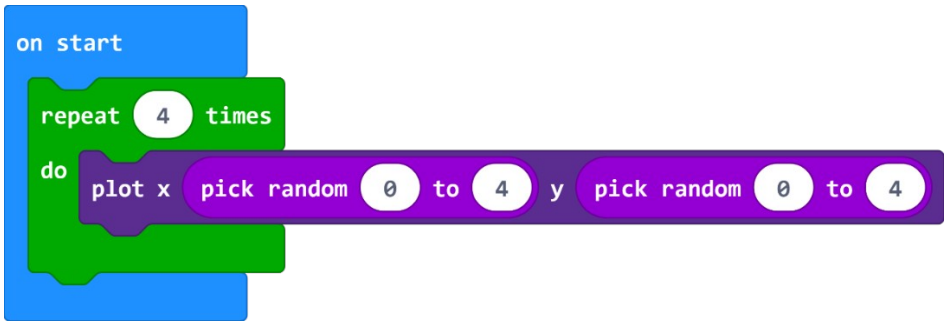
Code random constellations

1. In MakeCode, start a new project and name it something like ‘Starry night’. Either delete the ‘forever’ block in the coding Workspace or move it to the side as it’s not used in the activity.

Remember that the ‘pick random’ block in the Math Toolbox drawer will return a random value between a specified minimum and maximum value. You’ll use this to plot a single dot at a random location on the screen by choosing a random number from 0 to 4 for the x axis and a random number from 0 to 4 for the y axis.

2. From the Led Toolbox drawer, drag a ‘plot’ block and connect it inside the ‘on start’ block. Then, from the Math Toolbox drawer, drop a ‘pick random’ block into each of the x and y values and change the range in each from 0 to 4.

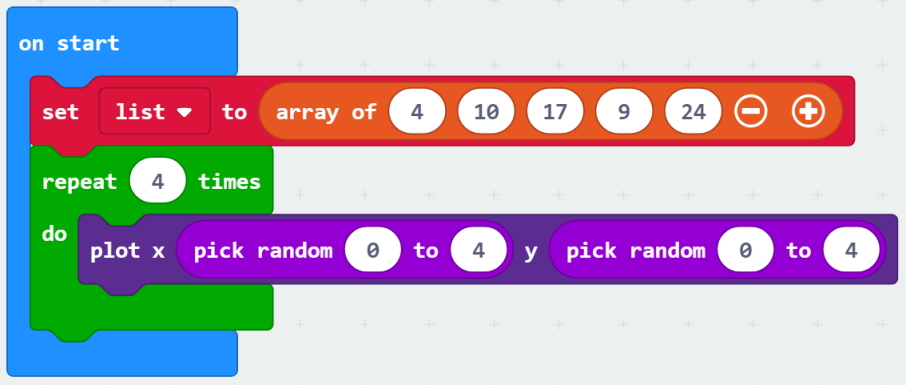
3. Next, create a loop that will repeat the above code four times to create a constellation with four stars using the ‘repeat’ block. To keep things simple, you won’t check for duplicates, so it’s possible you may end up with fewer than four visible stars. That’s okay.



Code the array

Next, create an array with five numbers in it. You will loop through the array and create five separate constellations using the numbers in the array to represent the number of stars in each of the five constellations.

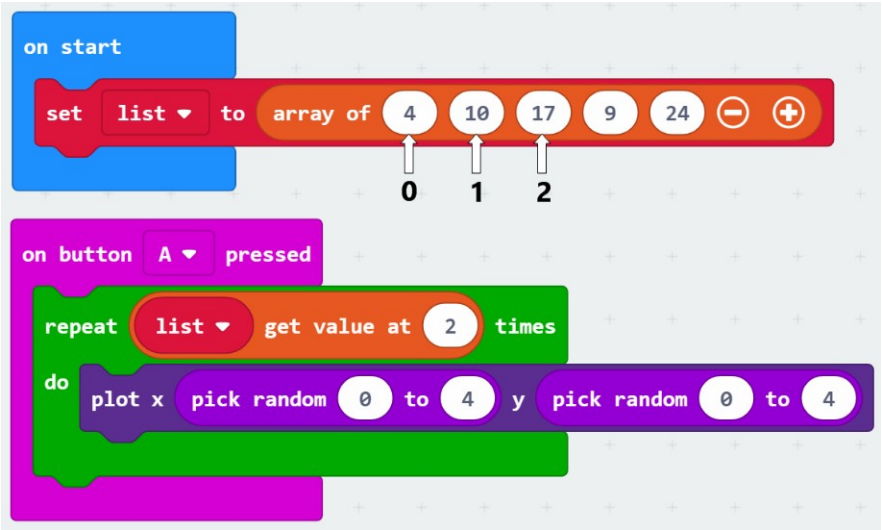
4. From the Arrays Toolbox drawer, drag out the ‘set list’ block onto the Workspace and drop it into the ‘on start’ block. Then, select the **(+)** symbol three times to add more values to the array for a total of five. Type five random numbers from 0 to 25 in your array list since there are up to 25 LED’s on the micro:bit that could be used in your constellation.



Code the A button

Activate your constellations when you press a button.

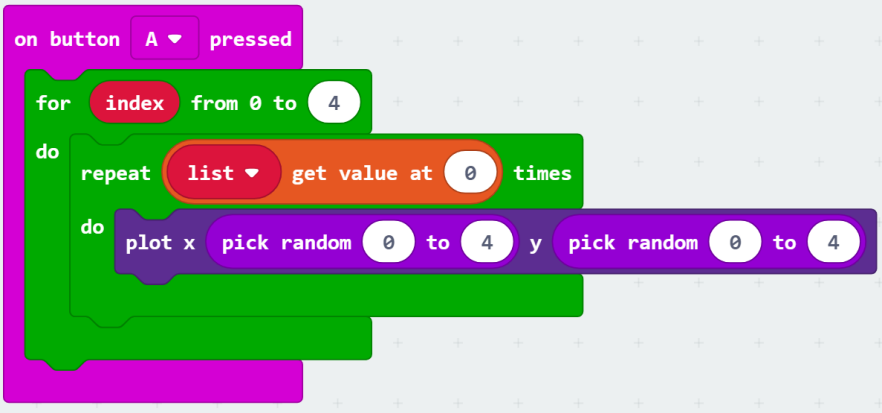
5. From the Inputs Toolbox drawer, drag out an ‘on button A pressed’ block onto the Workspace.
6. Drag the ‘repeat’ loop from the ‘on start’ block into the ‘on button A pressed’ block. Instead of repeating 4 times, use the values from the array to figure out how many stars to plot on your micro:bit.
7. From the Arrays Toolbox drawer, drag a ‘list get value at’ oval block onto the Workspace and drop it into the ‘repeat’ loop, replacing the default value of 4. When you type a 0, 1, 2, 3, or 4 into the ‘get value’ block, it will plot the number of stars indicated by the value held in the specified array index.



Loop through the array

Now, all you need to do is iterate through the entire array and do the same thing for each of the values. Instead of manually typing in values 0, 1, 2, 3, or 4 into the ‘list get value at’ block, use a variable and another loop to automatically increment the index each time.

8. From the Loops Toolbox drawer, drag another ‘for loop’ block onto the Workspace and drop it into the ‘on button A pressed’ block around the existing ‘repeat’ loop.



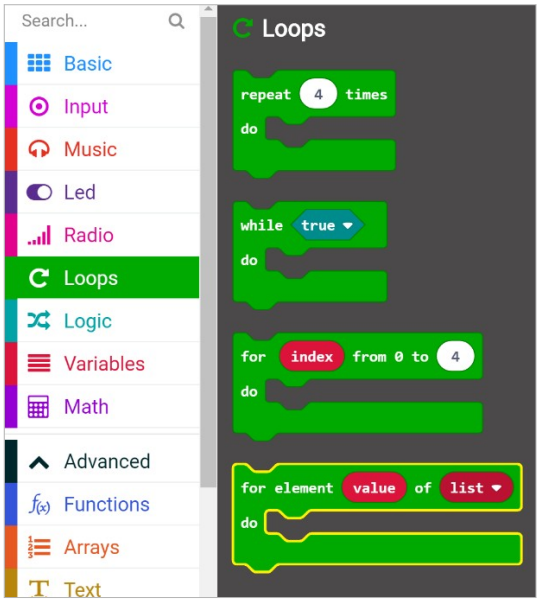
9. From the Variables Toolbox drawer, drag an ‘index’ variable block onto the Workspace and drop it into the ‘list get value at’ block replacing the default 0 value.

Test in the Simulator

10. Notice, that if you try this in the simulator, all the stars will be plotted at once because the loops run too fast for us to see. So, you’ll need to add a ‘pause’ block and a ‘clear screen’ block between each constellation.

Coding activity 3: Traversing an array

Traversing an array means proceeding through the elements of the array in sequence. You may have noticed that there is a special loop in the Loops Toolbox drawer called ‘for element value of list’. This loop automatically loops through each element of your array in turn to “traverse” through the array.



1. In Microsoft MakeCode, start a new project and name it something like ‘traversing arrays’. Either delete the ‘forever’ block in the coding Workspace or move it to the side as it’s not used in the activity.
2. Start a new project and create a simple array using the ‘for element’ loop block to traverse you array displaying each value.

Use this space to track your notes and ideas

Lesson C: Make a micro:bit musical instrument

Activity: micro:bit musical instrument

This is a project in which we challenge you to create a musical instrument that uses arrays to store sequences of notes. The array of notes can be played when an input occurs, such as one of the buttons being pressed or if one or more of the pins is activated.

Ideally, the micro:bit should be mounted in some kind of housing, perhaps a guitar shape or a music box. Start by looking at different kinds of musical instruments to get a sense of what kind of shape you might want to build around your micro:bit.

Project expectations

Follow the design thinking approach and make sure your project meets these specifications:

- Uses at least one array in a fully integrated and meaningful way and the array stores and iterates through each element of the array successfully
- The tangible instrument is tightly integrated with the micro:bit and each relies heavily on the other to make the project complete
- The program compiles and runs as intended and includes meaningful comments in code
- Provide the written Reflection Diary entry

The design thinking process:

1. Empathize by learning more about your target audience.
2. Define: Understand and identify your audience's problems or needs.
3. Ideate: Brainstorm several possible creative solutions.
4. Prototype: Construct rough drafts or sketches of your ideas.
5. Test your prototype solutions.

Use this space to track your design thinking process:

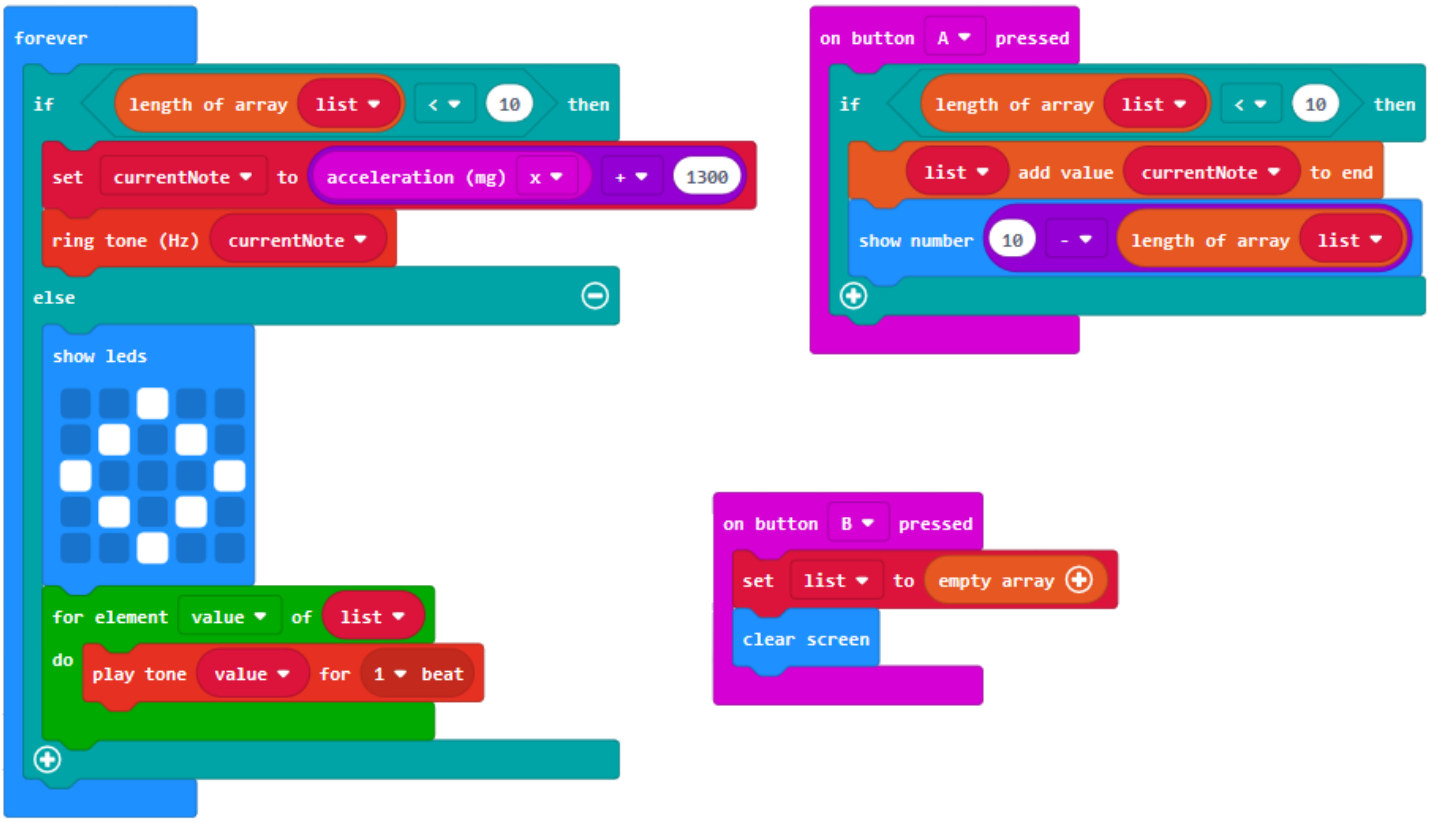
Project examples

micro:bit guitar

This project uses the micro:bit accelerometer to play different tones when the guitar is held and tilted while playing.

- Pressing the A button will save the current tone to an array.
- After ten tones, a repeating melody will be performed.
- Press the B button to clear the array and start over.

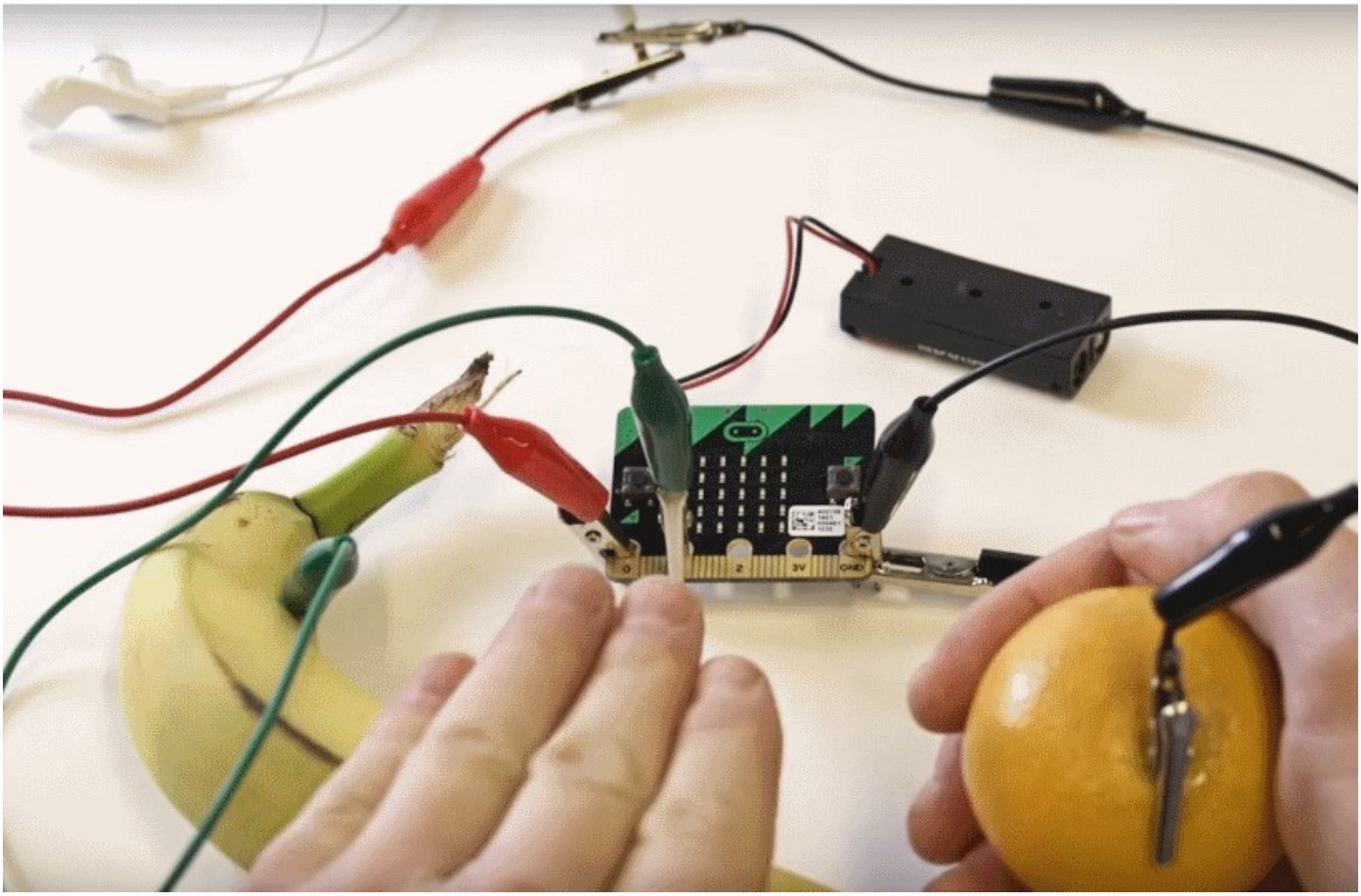
Complete program



Solution link: makecode.microbit.org/_546PpW8TYWJH

Banana keyboard

This project uses the pins to connect crocodile clips to pieces of fruit to activate the musical instrument.



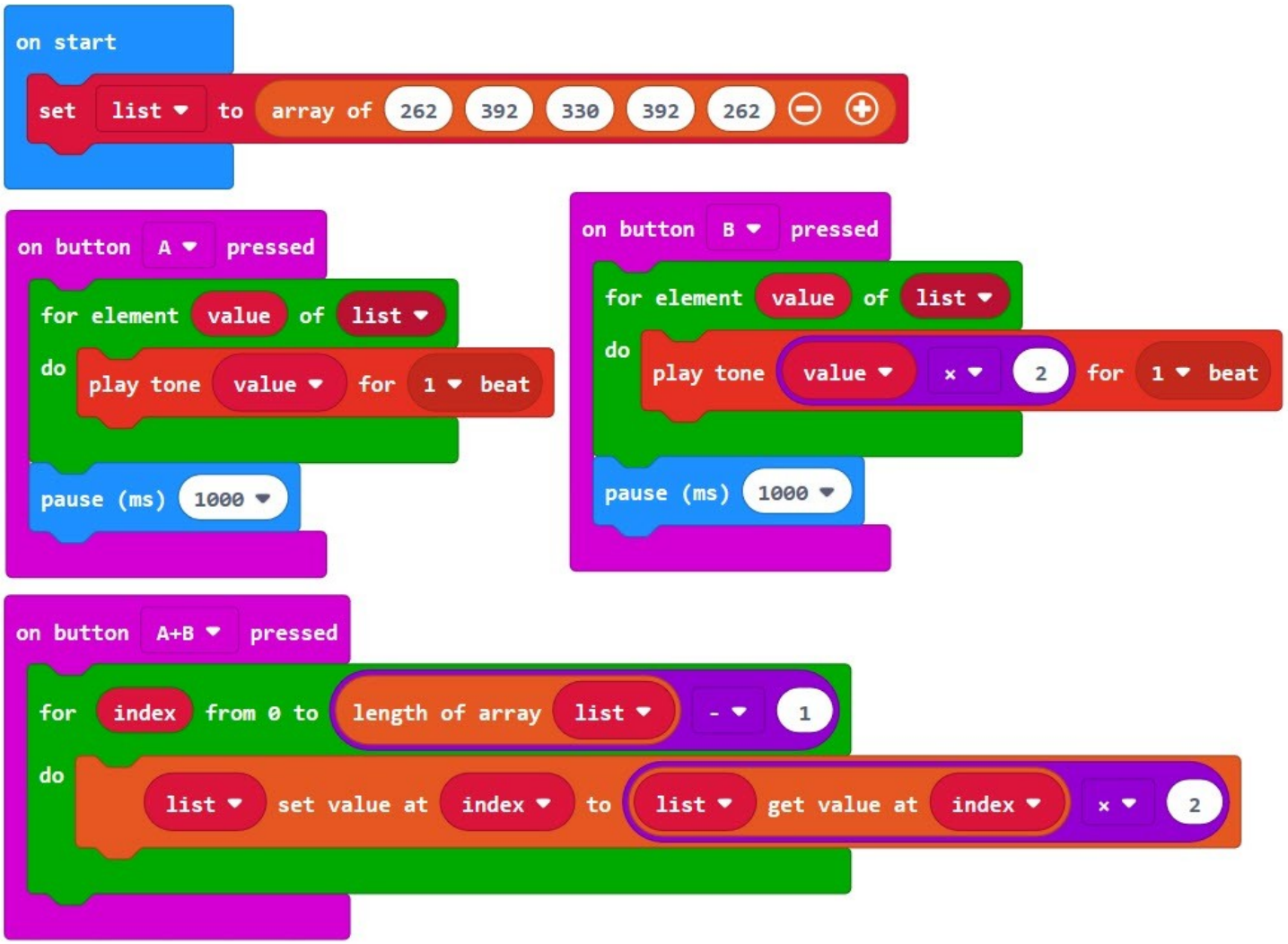
Using arrays with musical notes

You can create an array of notes by attaching Music blocks to an array. Musical notes are described in words (e.g., Middle C, High C, etc.), but they are actually numbers. You can do Math operations on those numbers to change the pitch of your song.

Here is an example of how to create an array with musical notes.

- Button A plays every note in the array.
- Button B plays the notes at twice the frequency (but doesn't alter the original notes.)

Complete code



Remember that a ‘for element (value) of list’ loop makes a temporary copy of the value, so even if you change a value, it will not change the original element in the array. If you want to permanently change the values in your array (to transpose music to increasingly higher keys, for example), you can use a for loop like this.

Solution link: makecode.microbit.org/_gwc1tveL1gfd

Project scoring rubric

Assessment elements	1	2	3	4
Array	Array doesn't work at all or no array present.	Array skips values or has other problems with storing and/or retrieving elements.	Stores each element of the array successfully.	Stores and iterates through each element of the array successfully.
Maker component	No tangible component.	Tangible component does not add to the functionality of the program.	Tangible component is somewhat integrated with the micro:bit but is not essential.	Tangible component is tightly integrated with the micro:bit and each relies heavily on the other to make the project complete.
micro:bit program	micro:bit program lacks three or more of the required elements.	Array is poorly implemented and/or peripheral to function of project, and/or lacks two of the required elements.	Uses an array in a tangential way that is peripheral to function of project and/or program lacks one of the required elements.	micro:bit program: ▪ Uses at least one array in a fully integrated and meaningful way ▪ Compiles and runs as intended ▪ Uses meaningful comments in code

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- Explain how you decided on your musical instrument. What was your inspiration instrument? What brainstorming ideas did you come up with?
- What properties does it share with a real musical instrument? What properties are unique?
- Describe the type of array you used (Numbers, Strings, or Notes) and how it functions in your project.
- What was something that was surprising to you about the process of creating this program?
- Describe a difficult point in the process of designing this program and explain how you resolved it.
- What feedback did your testers give you? How did that help you improve your musical instrument?
- Publish your MakeCode program and include the link.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements.

Glossary of key terms

Array: A list or collection of similar things. An object in an array is referred to as an element or item in the array. An individual element can be referenced by its index position in the array.

Array index: A unique address or location in the collection, e.g., page number in an album, shelf on a bookcase, etc.

Array length: The total number of items in the collection.

Array sort: How you could order items in the collection (for example: date, price, name, color, and so on). Well-known types of array sorts are Bubble Sort, Selection Sort, and Insertion Sort, although faster sorting algorithms like QuickSort and MergeSort are more commonly used to sort large data sets.

Array type: The type of item being stored in the collection, e.g., comics, $1 coins, Pokémon cards, numbers, strings, etc.